



1. Implement a multithreaded program where N threads (e.g., $N=4$) compete to update a shared global variable, `critical_limit`, which starts at 100. Each thread attempts to reduce `critical_limit` by 10 in a loop, but only if the current value is greater than 50. (20)

The critical operation is: Check if `critical_limit > 50`, and if true, subtract 10.

Write a C program implementing both Part (a) and Part (b) .

- a) Part a: Without Mutex (Observe Race Condition)

Implement the logic without any synchronization (i.e., no mutex). Run the program and record the final value of `critical_limit`.

Expected output:

```
--- Starting Part 1 (No Mutex) ---  
Initial Value: 100  
Thread 1 decremented. New value: ??  
Thread 4 decremented. New value: ??  
Thread 2 decremented. New value: ??  
Thread 3 decremented. New value: ??  
Thread 2 decremented. New value: ??  
Final Value (Unsafe): ??
```

- b) Part b: With Mutex (Observe Consistent Result)

Run the program and record the final value of `critical_limit`. This value should be 50 (since the last successful subtraction of 10 must occur when the value is 60).

Expected output:

```
--- Starting Part 2 (Mutex) ---  
Initial Value: 100  
Thread 1 decremented. New value: ??  
Thread 4 decremented. New value: ??  
Thread 2 decremented. New value: ??  
Thread 3 decremented. New value: ??  
Final Value (safe): ??
```

2. Write a C program that uses multiple threads to find the minimum and maximum values in a large integer array. (20)
- a) Create a large integer array (e.g., 1,000 elements) and fill it with random numbers.
- b) Divide the array into N segments, where N is the number of threads.

- c) Create N threads, with each thread responsible for finding the minimum and maximum values in its assigned segment.
- d) Each thread should return a structure containing the minimum and maximum values it found.
- e) The main function should collect the results from all threads and then find the overall minimum and maximum values from the partial results.
- f) The program should print the overall minimum and maximum values.

Note : This question requires you to combine concepts of thread creation, passing arguments, returning complex data structures from threads, and aggregating results in the main thread.

3. Write a C program using pthreads, mutex, and condition variables to implement an extended Writer-Priority Readers-Writers Problem. (20)
- a) There are 5 Readers and 3 Writers.
 - b) Each Reader performs 4 read operations, sleeping for a random time between 100–300 milliseconds between reads.
 - c) Each Writer performs 3 write operations, sleeping for a random time between 150–350 milliseconds between writes.
 - d) A shared integer variable `data_value` is updated only by writers.
 - e) The Writer Priority Rule ensures that no reader may enter if a writer is writing or waiting. Writers must get exclusive access.

The program output should include lines like:

```
Reader 2 is reading... Active Readers = 3 | Data Value = 17
...
Final Data Value = 9
Final Log Count = 38
```

4. Write a C program that simulates a simplified version of the Linux `tree` command using directory traversal. The program must recursively list the contents of a directory up to a user-specified depth (Depth represents the level of nesting of directories within directories; for example, if `depth = 2`, it means the `home` directory contains a subdirectory, and that subdirectory itself contains another directory.). The function `S_ISREG()` takes one argument returned by `stat()` or `lstat()` and returns true (non-zero) if the file is a regular file. Similarly, `S_ISLNK()` takes the file-mode value returned by `stat()` or `lstat()` and returns true (non-zero) if the file is a symbolic link. (20)

```
1 #include <stdio.h>
2 #include <dirent.h>
3 #include <string.h>
```

```

4  #include <sys/stat.h>
5  #include <stdlib.h>
6  #include <unistd.h>
7
8  void explore(const char *path, int depth, int level) {
9      if (level > depth)
10         return;
11     DIR *d = opendir(path);
12     if (!d) {
13         for (int i = 0; i < level * 4; i++)
14             printf(" ");
15         printf("%s [ACCESS DENIED]\n", path);
16         return;
17     }
18     struct dirent *de;
19     char newpath[512];
20     while ((de = readdir(d)) != NULL) {
21         if (!strcmp(de->d_name, ".") || !strcmp(de->d_name, ".."))
22             continue;
23         for (int i = 0; i < level * 4; i++)
24             printf(" ");
25         snprintf(newpath, sizeof(newpath), "%s/%s", path, de->
d_name);
26         struct stat st;
27         lstat(newpath, &st);
28         printf("%s", de->d_name);
29         if (S_ISDIR(st.st_mode))
30             printf(" [DIR]\n");
31         // ADD YOUR CODE
32     }
33     closedir(d);
34 }
35
36 int main(int argc, char *argv[]) {
37     if (argc != 3) {
38         printf("Usage: %s <directory> <depth>\n", argv[0]);
39         return 1;
40     }
41     int depth = atoi(argv[2]);
42     printf("%s\n", argv[1]);
43     if (depth > 0)
44         explore(argv[1], depth, 1);
45     return 0;
46 }

```

0 1 204
1 2 201

- a) Print File is a Directory, a Regular File, a Link or another type
- b) Call Recursively Directory for specified depth

Sample output might be:

On running

```
./a.out /home #represents output inside current directories  
./a.out /home/2 #represents output in two level directories inside home
```

```
. dtc [DIR] (size: 4096 bytes)  
Desktop [DIR] (size: 4096 bytes)  
.gitconfig [FILE] (size: 70 bytes)  
ticket1.pdf [FILE] (size: 565434 bytes)  
Videos [DIR] (size: 4096 bytes)  
Downloads [DIR] (size: 4096 bytes)  
Public [DIR] (size: 4096 bytes)
```
